SecTrace: Evolving Cross-Task Security Experience for Secure Code Generation

Anonymous submission

Abstract

Large language models are increasingly used to synthesize software, yet functionally plausible code can still contain exploitable weaknesses. Existing methods typically solve tasks independently and discard the security knowledge acquired during generation and repair, while directly reusing past solutions may propagate unreliable or context-misaligned guidance. To address these challenges, we introduce SecTrace, a multi-agent framework that treats cross-task security knowledge as revisable experience. For each task, SecTrace performs threat modeling, retrieves relevant experience to guide generation and repair, verifies candidates using dynamically constructed functional and security checks, and distills the resulting trace into reusable experience. The resulting task experience is stored in SafeMem, where it is iteratively consolidated, validated, and refined during an update phase before the memory is frozen for stable reuse. Across three LLM backbones and five benchmark settings, SecTrace achieves the best Pass@1 in 14 of 15 model–benchmark combinations. Further analyses demonstrate the complementary contributions of its core components, the benefit of controlled experience freezing, and the transferability of security experience across datasets and programming languages. These results highlight the importance of validating and refining cross-task security experience for more reliable secure code generation.

1 Introduction

Code generation aims to translate natural-language specifications into executable programs (Shin and Nam 2021). Recent advances in large language models (LLMs) have substantially improved automated program synthesis and accelerated the adoption of AI-assisted programming tools (Yang et al. 2024; Li et al. 2025; Klemmer et al. 2024). However, because LLMs are often trained on open-source repositories, they may also reproduce insecure coding patterns and vulnerable implementations. For example, Pearce et al. (2022) found that approximately 40% of Copilot-generated programs are insecure. Reliable AI-assisted programming therefore requires generated code to satisfy both functional correctness and security requirements.

Prior work has explored secure code generation from several perspectives. Prompt-based and retrieval-based approaches incorporate security instructions, vulnerability knowledge, or secure demonstrations into the generation process (Zhang et al. 2024). Training-based methods improve security awareness through instruction tuning, adversarial training, or preference optimization (He and Vechev 2023; He et al. 2024; Hasan et al. 2025). More recently, secure coding agents have decomposed the generation process into specialized roles for security analysis, code generation, verification, and targeted repair (Wang et al. 2026). Such agentic decomposition improves individual coding tasks by coordinating specialized roles and leveraging intermediate security feedback for iterative refinement.

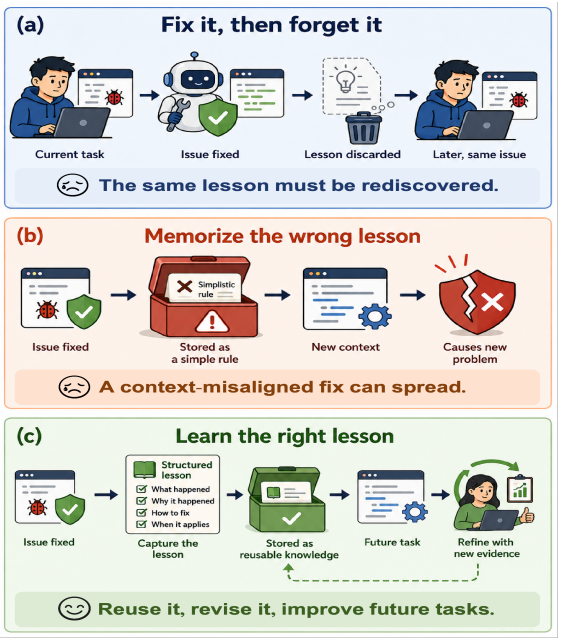


Figure 1: Motivation for evolving security experience. (a) Existing methods often discard useful security lessons afterward. (b) Naively reusing previous repairs as security experience may propagate context-misaligned knowledge. (c) Security experience should be preserved, reused, and refined.

Despite these advances, existing approaches still face two fundamental limitations. **First**, existing agents remain largely episodic and lack cross-task security experience accumulation. As illustrated in Figure 1(a), current secure coding agents can effectively analyze vulnerabilities and repair insecure programs within individual tasks. However, the security knowledge revealed during verification and repair

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

is typically discarded after task completion. Consequently, when future tasks encounter similar vulnerabilities, these agents must repeatedly rediscover previously identified security principles rather than benefiting from prior interactions. **Second**, naively treating previous repairs as reusable security experience can introduce new risks. As illustrated in Figure 1(b), unlike general coding experience, security-related knowledge is highly context-dependent and often relies on factors such as API semantics, data-flow assumptions, and threat models. A security fix that is effective in one context may become ineffective or overly restrictive in another setting. Moreover, inaccurate verification signals or incomplete tests may cause incorrect repairs to be stored and repeatedly reused, potentially propagating unreliable security knowledge across tasks.

To overcome these limitations, we propose **SecTrace**, a security-experience evolution framework that enables secure coding agents to accumulate, reuse, and refine security experience across tasks. As illustrated in Figure 1(c), SecTrace treats the security knowledge learned during code generation and repair as revisable experience rather than static memory. Specifically, SecTrace is built upon three core mechanisms. *First*, trace-grounded experience induction derives reusable security experience by analyzing how candidate programs evolve through generation, verification, and repair, rather than directly memorizing previous solutions or raw interaction histories. *Second*, experience-guided secure coding and repair retrieve relevant experiences from **SafeMem**, a shared memory that stores structured security experience, and use them as non-binding guidance, allowing agents to reuse accumulated security insights without blindly applying them in incompatible contexts. *Finally*, evidence-guided experience evolution treats each stored experience as a revisable hypothesis and updates its reliability and applicability using subsequent internal verification evidence. Similar experiences are consolidated to reduce redundancy, while contradictory evidence can decrease an experience's reliability score, narrow its applicability conditions, or trigger versioned correction and replacement. As shown in Figure 2, SecTrace realizes this closed-loop experience lifecycle through five role-specialized agents supported by SafeMem, covering threat modeling, experience-guided secure coding, dynamic test verification, experience-guided repair, and security experience evolution.

We evaluate SecTrace across five secure code generation benchmark settings with three LLM backbones, comparing it against prompting-based methods, general coding agents, security-oriented agents, and evolving-memory agents. The experimental results show consistent improvements over competitive baselines, demonstrating the effectiveness of SecTrace in enhancing secure code generation. In summary, our contributions are:

- We investigate *controlled security-experience evolution* for secure code generation, where cross-task security knowledge is not only accumulated and reused, but also continuously validated and revised. This formulation addresses the risks of context-misaligned reuse and unreliable knowledge accumulation when extending secure coding agents with cross-task memory.
- We propose SecTrace, a multi-agent framework that integrates trace-grounded experience induction, experience-guided secure coding and repair, and evidence-guided experience evolution. Specifically, SecTrace distills structured security experience from generation, verification, and repair processes, explicitly models its applicability, and revises it using subsequent internal evidence.
- We conduct extensive experiments across five secure code generation benchmark settings with various LLM backbones. The results demonstrate consistent improvements over competitive baselines, while additional ablation studies, experience-evolution analyses, and cross-dataset experience transfer experiments further validate the effectiveness and generality of SecTrace.

2 Related Work

2.1 Secure Code Generation with LLMs

Existing studies enhance the security of LLM-generated code by introducing security knowledge at different stages of the generation process. SecCoder retrieves task-relevant secure demonstrations and incorporates them into the generation context, improving the model's awareness of security requirements without updating its parameters (Zhang et al. 2024). Other approaches inject security knowledge into model parameters through parameter-efficient tuning or security-oriented instruction tuning (He and Vechev 2023; He et al. 2024). Hasan et al. (2025) further construct secure–insecure preference pairs and focus preference optimization on security-critical code regions. Moreover, agent-based methods use testing, execution feedback, static analysis, or self-debugging to identify and correct insecure candidates (Chen et al. 2024, 2025). Although these methods improve secure code generation through demonstrations, model adaptation, or verification-guided refinement, the security knowledge they use is generally predefined, fixed after training, or confined to the current task.

2.2 Experience-Driven LLM Agents

Experience-driven LLM agents improve their behavior by retaining and reusing information from previous interactions without updating model parameters. Reflexion stores verbal feedback in episodic memory to guide subsequent attempts (Shinn et al. 2023), whereas ExpeL extracts reusable natural-language insights from collections of successful and failed trajectories (Zhao et al. 2024). Voyager maintains an expanding library of executable skills that can be retrieved and composed for new embodied tasks (Wang et al. 2024), while ReMe investigates procedural-experience distillation and context-adaptive retrieval across tasks (Cao et al. 2026). These studies demonstrate that external memory can reduce repeated trial and error. Nevertheless, existing methods primarily emphasize the accumulation and reuse of past experience, while providing limited support for continuously validating its reliability and applicability using subsequent evidence. In particular, they do not specifically address how unreliable security guidance should be conservatively corrected and replaced across code generation tasks.

3 Methodology

3.1 Framework Overview

Given a sequence of code generation tasks $\mathcal{T} = \{\tau_1, \tau_2, \ldots, \tau_N\}$, where each task τ_t provides a natural-language or docstring-style specification, SecTrace accesses the security experience memory $\mathcal{M}_{t-1}$ accumulated from previous tasks before processing τ_t. It aims to produce a final program c_t^* that satisfies both functional correctness and security requirements, while using the resulting generation, verification, and repair trace to evolve the security experience available to subsequent tasks.

As illustrated in Figure 2, SecTrace consists of five role-specialized agents supported by SafeMem. First, the *Threat Modeling Agent* analyzes the task specification and constructs a structured threat analysis of the task. Next, the *Experience-Guided Secure Coding Agent* retrieves relevant experience from SafeMem and uses it to generate an initial program. The *Dynamic Test Verification Agent* then synthesizes and executes functional and security tests to obtain execution feedback. When verification fails, the *Experience-Guided Repair Agent* retrieves failure-relevant experience and performs a targeted repair. Finally, the *Security Experience Evolution Agent* distills reusable security lessons from the complete generation–verification–repair trace and updates SafeMem for subsequent tasks.[1]

Each security experience $e \in \mathcal{M}$ consists of a retrieval state s_e and guidance content g_e. The retrieval state s_e describes *when an experience is applicable and how reliable it is for reuse*. The retrieval state encodes the experience's applicability through six security-relevant dimensions: security risk type, security-sensitive operation, untrusted input source, security-sensitive sink, source-to-sink flow or API usage, and applicability conditions. It also records the experience's lifecycle status (TRIAL or CONFIRMED) and evidence-based reliability score q_e, which summarizes the supporting and contradictory evidence accumulated across tasks. During retrieval, the six security-relevant dimensions, lifecycle status, and reliability score jointly determine whether an experience is eligible for selection and how highly it is prioritized. In contrast, the guidance content g_e specifies *how the experience should guide coding decisions*. It includes the triggering condition, the security invariant to preserve, the recommended action, the unsafe anti-pattern to avoid, and the exclusion conditions under which the experience should not be applied. Together, these elements provide actionable but non-binding guidance for code generation and repair.

3.2 Threat Modeling

Secure code generation requires more than translating a functional specification into syntactically valid code. A model must first determine which inputs may be untrusted, how they may propagate through the program, which operations are security-sensitive, and which security properties must be preserved. To make these security semantics explicit, the *Threat Modeling Agent* analyzes the task specification, function signature, and available code context before code generation. It constructs a structured threat analysis as follows:

$$z_t = \mathcal{H}(\tau_t) \tag{1}$$

where $\mathcal{H}$ denotes the task-adaptive threat modeling process. The resulting z_t summarizes potential risks, security-sensitive operations, untrusted sources and sensitive sinks, source-to-sink data flows, and relevant secure or unsafe implementation patterns. It provides a task-specific security basis for subsequent experience retrieval, code generation, dynamic verification, and repair.

3.3 Experience-Guided Secure Coding

The structured threat analysis enables SecTrace to retrieve previously accumulated security experience according to the security semantics of the current task. Given the task τ_t, its threat analysis z_t, and the preceding memory $\mathcal{M}_{t-1}$, the *Experience-Guided Secure Coding Agent* retrieves a set of relevant experiences from SafeMem:

$$\mathcal{E}_t^G = \mathcal{R}_G\big(\tau_t, z_t, \mathcal{M}_{t-1}\big) \tag{2}$$

where $\mathcal{R}_G$ denotes the experience retrieval process for initial code generation. SecTrace adopts a conservative retrieval strategy to reduce context-misaligned experience reuse. Retrieval first filters out experiences whose exclusion conditions, API assumptions, or security-sensitive flows conflict with the current threat analysis, and ranks the remaining experiences using semantic relevance, lifecycle status, and reliability score. For each experience e, the retrieval process compares the task-specific threat analysis z_t with the six security-relevant dimensions encoded in its retrieval state s_e. The resulting relevance is combined with the experience's lifecycle status and reliability score to determine its retrieval priority. In this process, CONFIRMED experiences that have accumulated stronger support from previous tasks are prioritized, whereas TRIAL experiences require stricter matching.

The guidance content g_e of each retrieved experience is then provided to the agent as a non-binding prior to generate the initial candidate program:

$$c_t^{(0)} = \mathcal{G}\big(\tau_t, z_t, \mathcal{E}_t^G\big) \tag{3}$$

where $\mathcal{G}$ denotes the experience-guided coding process.

3.4 Dynamic Test Verification

The *Dynamic Test Verification Agent* constructs a functional test suite $\mathcal{Q}_t^F$ and a security test suite $\mathcal{Q}_t^S$ from the task specification τ_t and threat analysis z_t, which together form the dynamically generated test suite $\mathcal{Q}_t$. The generated tests are used only for SecTrace's internal verification and repair, whereas the official benchmark tests and all associated feedback are reserved exclusively for external evaluation and are never exposed to any agent or SafeMem. Functional tests examine the intended program semantics, whereas security tests operationalize the potential risks and sensitive behaviors identified during threat modeling. To improve test reliability, SecTrace adopts a voting mechanism for both functional and security test construction. For each generated input, the

[1] Due to space constraints, some detailed implementation configurations, including agent interaction protocols, retrieval strategies, and memory evolution procedures, are provided in the supplementary material.

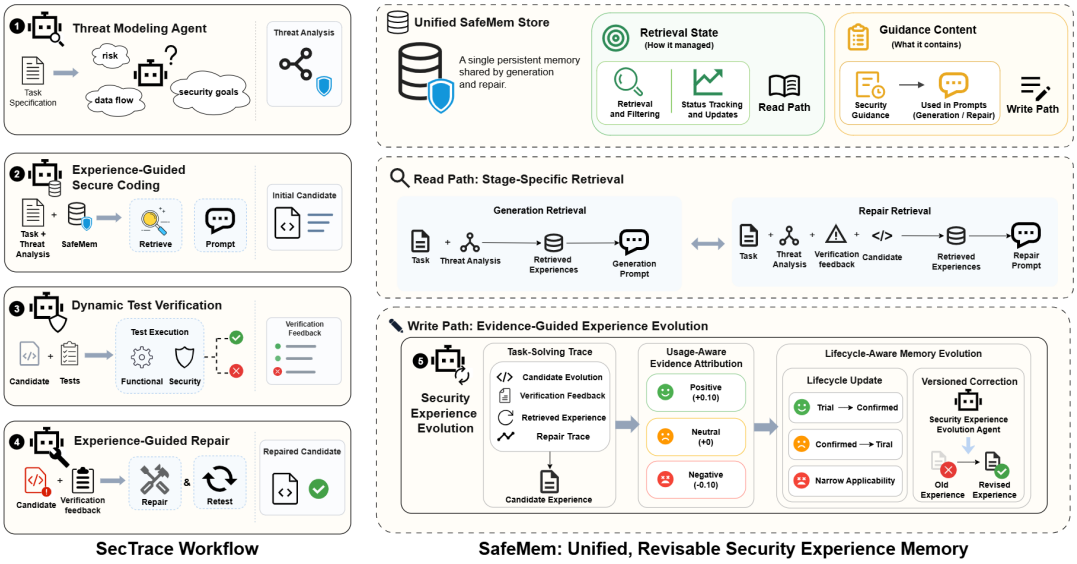


Figure 2: Overview of SecTrace. The left panel shows the five-agent task-solving workflow, while the right panel illustrates SafeMem retrieval, evidence attribution, lifecycle transitions, consolidation, and versioned correction.

agent produces two independently sampled outcome predictions under the same prompt and backbone using different decoding seeds. The test case is retained when the predictions agree, otherwise, a third prediction is generated and the expected outcome is determined by majority agreement. Cases without a consistent outcome are discarded.

Once constructed, $\mathcal{Q}_t$ is reused throughout the verification–repair process. At verification iteration k, the current candidate $c_t^{(k)}$ is executed against the test suite:

$$r_t^{(k)} = \mathcal{V}\big(c_t^{(k)}, \mathcal{Q}_t\big) \tag{4}$$

where $\mathcal{V}$ denotes test execution and $r_t^{(k)}$ records the outcomes of functional and security tests, including observed outputs and runtime exceptions. If all tests pass, the candidate is accepted as the final program c_t^*. The final output for task τ_t is fixed before its trace is used to update SafeMem, so experience induced from the current task can affect only subsequent tasks. Otherwise, the failed cases and execution evidence are passed to the repair agent.

3.5 Experience-Guided Repair

When verification fails, the *Experience-Guided Repair Agent* retrieves failure-relevant experiences from SafeMem using the current candidate $c_t^{(k)}$, verification feedback $r_t^{(k)}$, and threat analysis z_t:

$$\mathcal{E}_{t,k}^{R} = \mathcal{R}_R\big(\tau_t, z_t, c_t^{(k)}, r_t^{(k)}, \mathcal{M}_{t-1}\big) \tag{5}$$

where $\mathcal{R}_R$ denotes the repair-oriented experience retrieval process. Compared with the retrieval process used for initial generation, repair retrieval additionally considers the current implementation and observed failure patterns.

The retrieved experiences are provided as non-binding guidance together with the failed test cases and execution evidence to guide repair of the current candidate:

$$c_t^{(k+1)} = \mathcal{P}\big(\tau_t, c_t^{(k)}, r_t^{(k)}, \mathcal{E}_{t,k}^{R}\big) \tag{6}$$

where $\mathcal{P}$ denotes the targeted repair process. The repaired candidate is subsequently re-evaluated against the same dynamically generated test suite. This verification–repair process continues until the candidate passes all generated tests or the maximum number of repair iterations is reached.

3.6 Security Experience Evolution

After the verification–repair process terminates, the *Security Experience Evolution Agent* distills reusable lessons from the complete task-solving trace and revises their reliability and applicability using execution-grounded evidence.

Trace-grounded experience induction. For task τ_t, the complete task-solving trace Γ_t is represented as:

$$\Gamma_t = \left\langle \tau_t, z_t, \left\{ \big(c_t^{(k)}, r_t^{(k)}\big) \right\}_{k=0}^{K_t}, \mathcal{E}_t^{\text{ret}} \right\rangle \tag{7}$$

where K_t denotes the final verification iteration, and $\mathcal{E}_t^{\text{ret}}$ denotes the experiences retrieved during initial generation, denoted by $\mathcal{E}_t^{G}$, and across subsequent repair iterations, denoted by $\mathcal{E}_t^{R}$. The trace captures the evolution of candidate programs, their functional and security outcomes, and the guidance provided by retrieved experiences.

Based on Γ_t, the agent distills a set of candidate experiences, denoted by $\widetilde{\mathcal{E}}_t$. When the initial candidate passes verification, the agent extracts security decisions supported by the generated tests. When repair occurs, it contrasts the failed and repaired candidates together with their execution outcomes, identifying the unsafe decision that caused the failure and the correction that resolved it. Each candidate experience is represented using the retrieval state and guidance content defined in Section 3.1, and is subsequently consolidated with an existing experience or admitted as a new one.

Usage-aware evidence attribution. Retrieval alone does not establish that an experience is reliable. For each retrieved experience e, SecTrace determines whether its guidance is

reflected in the candidate and supported by subsequent verification. The task-level evidence assigned to e is:

$$\delta_t(e) = \mathcal{A}\big(e, \Gamma_t\big) \in \{-0.1, 0, +0.1\} \tag{8}$$

where $\mathcal{A}$ denotes the usage-aware evidence attribution process. For each retrieved experience, SecTrace derives a validated usage signal from implementation-level guidance anchors or model-reported applied-experience identifiers. It then combines this signal with the before-and-after internal verification results and the final candidate decision to record a structured attribution event, indicating whether the experience was not used, supported, contradicted, or left unresolved by the available internal execution evidence. Positive evidence is assigned when applied guidance is supported by verification. Negative evidence is assigned when its applied guidance is contradicted by execution outcomes or fails to resolve the targeted defect. Neutral evidence is assigned when the experience is not reflected in the implementation or when the trace provides insufficient evidence. This mechanism prevents retrieved but unused experiences from being indiscriminately reinforced.

Lifecycle-aware memory evolution. Using the attributed task-level evidence, SecTrace jointly updates the reliability scores and lifecycle statuses of stored experiences:

$$\begin{aligned} q_e^{(t)} &= \mathcal{F}\big(q_e^{(t-1)}, \delta_t(e)\big), \quad \forall e \in \mathcal{E}_t^{\mathrm{ret}} \\ \mathcal{M}_t &= \mathcal{U}\left(\mathcal{M}_{t-1}, \widetilde{\mathcal{E}}_t, \{\delta_t(e)\}_{e \in \mathcal{E}_t^{\mathrm{ret}}}\right) \end{aligned} \tag{9}$$

Here, $q_e^{(t)}$ denotes the reliability score of experience e after task τ_t, and $\mathcal{F}$ computes its updated value by aggregating the current evidence with the previously accumulated score.

The memory update process $\mathcal{U}$ handles the consolidation or admission of newly distilled experiences, and commits the updated reliability scores, lifecycle statuses, applicability conditions, and versioning relations to SafeMem. SecTrace further introduces an experience-update fraction hyperparameter $\rho \in [0, 1]$. SafeMem remains updatable during the first $\lfloor \rho N \rfloor$ tasks and is then frozen, while experience retrieval remains enabled. This controlled freezing limits the accumulation of redundant or context-misaligned experience and stabilizes its reuse in later tasks.

For each newly distilled experience, SecTrace first retrieves similar experiences from SafeMem according to their retrieval states and guidance content. If a compatible experience is found, the new experience is consolidated into the existing one. The existing experience retains its lifecycle status and reliability score. Applicability is broadened only when supported by independent positive evidence, exclusion conditions are accumulated conservatively, and candidates with unresolved semantic conflicts are not consolidated. Otherwise, the candidate is admitted as a new TRIAL experience with reliability score 0.30. A TRIAL experience is promoted to CONFIRMED only after receiving positive evidence from at least three distinct subsequent tasks and reaching a reliability score of 0.60. A CONFIRMED experience is demoted to TRIAL when its reliability score falls below 0.50 or when it receives contradictory evidence from at least two distinct subsequent tasks.

For experiences that repeatedly receive contradictory evidence, SecTrace performs versioned correction. When an experience applied during repair is associated with the same security-sensitive failure pattern across two independent tasks, the agent analyzes the original experience, the failed repair, the failed candidate, the final accepted candidate, and the intervening repair trace. A narrower revised version is created only when the final candidate resolves the targeted failure without introducing new failures. The revision preserves stable retrieval state and guidance fields while updating those supported by the observed correction, such as the recommended action, unsafe anti-pattern, and exclusion conditions. Moreover, the revised experience is initialized with the TRIAL status, thereby limiting its retrieval priority until it accumulates further supporting evidence. After versioned correction, the original experience is excluded from standard retrieval. In this way, SafeMem continuously consolidates, validates, and corrects cross-task security experience.

4 Experiments

4.1 Experimental Setup

Research questions. We evaluate SecTrace on widely used secure code generation benchmarks by addressing the following research questions:

- **RQ1:** How effective is SecTrace for secure code generation compared to existing approaches?
- **RQ2:** How do SecTrace's core components contribute to overall performance?
- **RQ3:** How does the experience update fraction affect the performance of SecTrace?
- **RQ4:** Can experience learned from one dataset transfer effectively to other datasets and programming languages?

Benchmarks. We evaluate SecTrace on five settings from three secure code generation benchmarks: CWEval with 119 tasks (Peng et al. 2025); the Python and Java code-completion settings of SecCodePLT with 819 and 924 tasks, respectively (Nie et al. 2025); and SecEvalBase and SecEvalPlus from CodeSecEval with 115 and 140 Python tasks, respectively (Wang et al. 2026). Unlike conventional functional benchmarks such as HumanEval (Chen et al. 2021) and MBPP (Austin et al. 2021), or earlier security benchmarks such as SecurityEval (Siddiq and Santos 2022) and CyberSecEval (Bhatt et al. 2023), which primarily assess vulnerability presence through static-analysis evaluation, these settings enable more precise automatic evaluation by providing paired executable functional and vulnerability-targeted security tests.

Metrics. Following CodeSecEval (Wang et al. 2026), we use Pass@1 as the primary metric, measuring the percentage of tasks whose final program passes both functional and security tests. We also report F-Pass@1 and S-Pass@1, computed on the functional and security test subsets, respectively, with complete results provided in the supplementary material.

Baselines and LLM backbones. We compare SecTrace against ten baselines, including four prompting methods: Greedy, Greedy with the Secure Prompt, Chain-of-Thought (CoT) (Wei et al. 2022), and CoT with the Secure Prompt; five

agent methods: AutoSafeCoder (Nunez et al. 2024), RA-Gen (Liu et al. 2025), AgentCoder (Huang et al. 2023), SecAware-Coder (Wang et al. 2026), and mini-SWE-agent (SWE-agent Team 2025); and the memory baseline ReMe (Cao et al. 2026). The main comparison uses Gemini-2.5-Flash (Comanici et al. 2025), DeepSeek-V4-Flash (DeepSeek-AI 2026), and GPT-5.1 (OpenAI 2025), while subsequent analyses exclude GPT-5.1 due to inference cost. Within each method–backbone comparison, we use the same backbone version and align decoding configurations, task orders, and maximum repair iterations whenever supported by the corresponding method. For each method–backbone–benchmark combination, we conduct five runs using independently shuffled task orders. The same five task orders are shared across all methods within each benchmark, and we report the mean performance across the five runs. Each SecTrace run processes tasks sequentially with an initially empty SafeMem. Except for the experience-update-fraction study, we use the same fixed value $\rho = 0.5$ across all benchmarks without per-dataset tuning.[2]

4.2 Main Results (RQ1)

As shown in Table 1, SecTrace achieves the best Pass@1 in 14 out of 15 backbone–benchmark settings and consistently outperforms the experience-driven ReMe baseline across all settings. Compared with the strongest baseline in individual settings, SecTrace improves Pass@1 by 12.55 points on PLT-Java with DeepSeek-V4-Flash, 9.26 points on CWEval with Gemini-2.5-Flash, and 5.87 points on CSE-Plus with Gemini-2.5-Flash. Moreover, its advantage over ReMe reaches 17.52 points on CWEval with GPT-5.1. These results indicate that effective cross-task reuse requires more than simply storing and retrieving prior experience: the reliability and applicability of stored guidance must be continually evaluated and revised as new evidence accumulates.[3][4]

Furthermore, all methods perform poorly on CSE-Base, where the highest Pass@1 is only 6.16%. Our qualitative analysis of failed CSE-Base cases indicates that many failures involve underspecified external dependencies, including API responses, database schemas, request contexts, and file fixtures. Nevertheless, SecTrace achieves the best CSE-Base result under all three backbones. It also consistently leads on CSE-Plus, outperforming the strongest baselines by 1.43–5.87 points across the three backbones. Overall, these results answer RQ1 positively: SecTrace delivers stronger and more robust secure code generation across LLM backbones, programming languages, and benchmark formulations.[5]

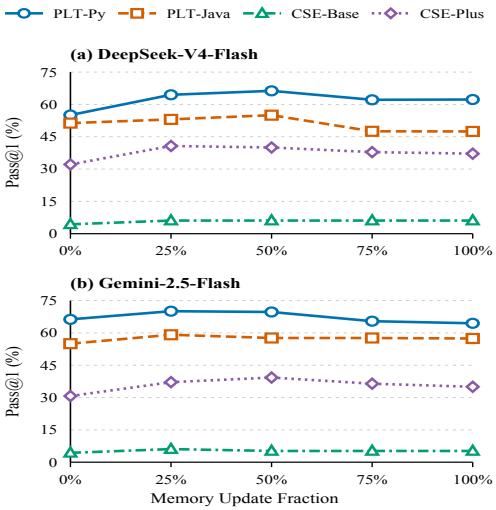


Figure 3: Results under different experience-update fractions.

4.3 Ablation Study (RQ2)

As shown in Table 2, removing any component reduces Pass@1, while the complete SecTrace achieves the best result in every setting. Dynamic test verification is particularly important for execution-sensitive tasks, with drops of 14.62 points on PLT-Java with DeepSeek-V4-Flash and 7.14 points on CSE-Plus with Gemini-2.5-Flash. Removing SafeMem also causes substantial degradation, including 11.10 points on PLT-Py with DeepSeek-V4-Flash and 8.57 points on CSE-Plus with Gemini-2.5-Flash, confirming the value of cross-task experience reuse.

Experience evolution also provides substantial gains, particularly on PLT-Py and CSE-Plus with Gemini-2.5-Flash, where its removal reduces Pass@1 by 8.54 and 5.14 points, respectively. This shows that merely storing experience without continuously revising its reliability and applicability is insufficient for effective reuse. Threat modeling benefits all evaluated settings, although its impact varies across backbones, suggesting that different LLMs differ in their ability to infer security constraints directly from task specifications. Overall, these results answer RQ2 positively: SecTrace's components provide distinct and complementary benefits, and their integration enables more reliable secure coding.

4.4 Effect of Experience-Update Fraction (RQ3)

To answer RQ3, we vary the experience-update fraction $\rho \in \{0, 0.25, 0.5, 0.75, 1.0\}$. Here, $\rho = 0$ disables cross-task updates, whereas $\rho = 1$ keeps SafeMem writable throughout the task stream. Once the specified fraction is reached, SafeMem is frozen while retrieval remains enabled.

Figure 3 shows a non-monotonic relationship between the update fraction and Pass@1. Updating SafeMem during the

[2]Detailed prompts, configurations, and token consumptions are provided in the supplementary material.

[3]The supplementary material provides a detailed case study of SafeMem's evidence-guided evolution, including experience consolidation, reliability-score updates, lifecycle transitions, and versioned correction. We also randomly sample 30 experiences from each benchmark and categorize them as *correct*, *incorrect*, or *over-generalized* to assess the quality of the evolved security experience.

[4]The supplementary material also includes a manual analysis in which we randomly sample 30 experiences from each benchmark and categorize them as *correct*, *incorrect*, or *over-generalized* to assess the quality of the evolved security experience.

[5]We further report F-Pass and S-Pass in the supplementary material to analyze the source of the overall improvements. The results show that SecTrace improves security correctness while preserving functional correctness, indicating that the gains are not achieved by overly conservative solutions or sacrificing task completion ability.

Table 1: Pass@1 across three LLM backbones and five benchmark settings. PLT-Py/Java denote SecCodePLT settings, and CSE-Base/Plus denote CodeSecEval subsets. "–" denotes unavailable PLT-Java results due to Python-only or tooling.

	GPT-5.1					DeepSeek-V4-Flash					Gemini-2.5-Flash				
Method	CWEval	PLT-Py	PLT-Java	CSE-Base	CSE-Plus	CWEval	PLT-Py	PLT-Java	CSE-Base	CSE-Plus	CWEval	PLT-Py	PLT-Java	CSE-Base	CSE-Plus
Prompting-based Methods															
Greedy	58.12	57.18	52.44	4.41	30.08	56.21	57.46	27.54	3.42	25.78	49.66	53.04	23.12	3.55	16.36
Greedy + Secure	60.42	57.95	54.15	4.32	27.92	66.47	57.44	32.84	3.55	27.06	59.58	55.63	36.72	4.28	15.08
CoT	60.56	59.38	51.24	3.56	29.22	55.39	56.11	28.41	2.58	25.64	46.31	52.56	34.26	3.42	12.22
CoT + Secure	65.47	57.70	53.41	4.38	22.79	68.14	54.16	32.84	3.42	25.79	52.02	54.29	34.06	2.66	14.93
Agent- and Memory-based Methods															
AutoSafeCoder	25.34	51.46	–	4.31	28.64	49.66	42.08	–	1.79	22.78	46.15	51.61	–	3.41	22.20
RA-Gen	27.81	68.43	51.03	2.58	30.64	67.17	65.52	34.36	2.67	23.49	31.18	65.25	44.84	1.79	16.51
AgentCoder	73.04	74.29	–	1.81	27.06	65.61	**70.26**	–	1.79	27.22	52.18	41.08	–	2.63	20.78
SecAwareCoder	47.14	55.49	–	2.67	15.07	62.11	50.24	–	2.55	22.06	37.05	43.39	–	2.58	17.94
mini-SWE-agent	40.27	62.28	47.91	4.29	17.79	52.86	60.74	40.08	4.41	25.08	50.49	67.44	48.56	3.55	20.64
ReMe	59.72	72.34	61.27	3.42	42.79	69.82	57.55	42.53	1.79	35.08	66.31	60.88	57.36	1.81	33.49
SecTrace	**77.24**	**74.91**	**63.48**	**6.14**	**44.22**	**75.71**	66.24	**55.08**	**6.16**	**40.07**	**75.57**	**69.79**	**57.71**	**5.18**	**39.36**

Table 2: Ablation results. "Threat Model," "Dynamic Test," and "Exp. Evolution" denote threat modeling, dynamic test verification, and security experience evolution, respectively.

Variant	PLT-Py	PLT-Java	CSE-Base	CSE-Plus
DeepSeek-V4-Flash				
w/o Threat Model	59.41	52.76	2.67	35.08
w/o Dynamic Test	60.73	40.46	3.55	37.79
w/o Exp. Evolution	60.86	53.46	6.02	37.92
w/o SafeMem	55.14	51.25	4.41	32.22
SecTrace	**66.24**	**55.08**	**6.16**	**40.07**
Gemini-2.5-Flash				
w/o Threat Model	69.52	56.46	4.28	35.64
w/o Dynamic Test	67.21	56.01	3.55	32.22
w/o Exp. Evolution	61.25	54.04	5.10	34.22
w/o SafeMem	66.21	55.08	4.41	30.79
SecTrace	**69.79**	**57.71**	**5.18**	**39.36**

early task stream improves performance over $\rho = 0$ across all evaluated settings, confirming the general utility of cross-task experience. Moderate fractions ($\rho = 0.25$ or 0.5) typically perform best, whereas continued updating beyond this stage tends to reduce performance. This suggests that useful experience accumulates early, while prolonged updating may introduce redundant or context-misaligned guidance.

The optimal fraction varies across backbones and datasets, likely because the same fraction corresponds to different numbers of update tasks and evidence availability. Therefore, $\rho = 0.5$ serves as a robust compromise rather than a universally optimal value. Overall, these results answer RQ3: controlled freezing preserves reusable experience while reducing instability from continuous memory updates.

4.5 Cross-Dataset Experience Transfer (RQ4)

To answer RQ4, SecTrace updates SafeMem on the first 50% of a source task stream and transfers the frozen memory to a different target setting. Retrieval remains enabled on the target, but no target-specific updates are performed. We compare transferred experience with both an empty SafeMem and in-domain experience learned from the target setting.

As shown in Table 3, transferred experience generally outperforms the no-memory setting. In particular, experience obtained from the larger PLT datasets improves all transfers to CSE-Plus by 2.85–8.40 points. Transferred experience can also occasionally exceed in-domain experience, such as PLT-Py → CSE-Plus with DeepSeek-V4-Flash and PLT-Java → CSE-Base with Gemini-2.5-Flash. This suggests that a larger source stream may provide richer security experience than a limited target-specific update stream, although the inconsistent gains on CSE-Base indicate that transferred experience cannot fully address underspecified external dependencies.

Table 3: Cross-dataset transfer results. $\Delta_{\text{no-mem}}$ denotes the change over SecTrace without SafeMem, while $\Delta_{\text{in-domain}}$ denotes the change over SecTrace using target-specific experience.

	DeepSeek-V4-Flash			Gemini-2.5-Flash		
Source → Target	Pass@1	$\Delta_{\text{no-mem}}$	$\Delta_{\text{in-domain}}$	Pass@1	$\Delta_{\text{no-mem}}$	$\Delta_{\text{in-domain}}$
PLT-Py → PLT-Java	52.08	$+0.83$	-3.00	59.76	$+4.68$	$+2.05$
PLT-Py → CSE-Base	6.13	$+1.72$	-0.03	4.41	0.00	-0.77
PLT-Py → CSE-Plus	40.62	$+8.40$	$+0.55$	33.64	$+2.85$	-5.72
PLT-Java → PLT-Py	63.87	$+8.73$	-2.37	66.08	-0.13	-3.71
PLT-Java → CSE-Base	4.39	-0.02	-1.77	6.91	$+2.50$	$+1.73$
PLT-Java → CSE-Plus	37.22	$+5.00$	-2.85	36.36	$+5.57$	-3.00

Cross-language transfer is also generally effective. Python experience improves Java under both backbones, while Java experience substantially improves Python with DeepSeek-V4-Flash. Although transferred experience underperforms in-domain experience in most settings, it still provides useful improvements without target-specific updates. Overall, RQ4 is answered positively: SafeMem captures security knowledge transferable across datasets and languages, while target-specific experience generally remains more reliable.

5 Conclusion

This paper presents SecTrace, a multi-agent framework that treats cross-task security knowledge as revisable experience. By integrating threat modeling, internal dynamic verification, experience-guided generation and repair, and evidence-based memory evolution, SecTrace improves Pass@1 across multiple backbones and benchmark settings. The results further show that controlled memory freezing can stabilize reuse and that some security experience transfers across datasets and programming languages. However, the current evaluation focuses on function-level tasks, and extending experience evolution to repository-level development remains challenging. Future work will explore adaptive memory evolution strategies, stronger verification mechanisms, and repository-level extensions to develop more capable coding agents.

References

Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q. V.; and Sutton, C. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732*.

Bhatt, M.; Chennabasappa, S.; Nikolaidis, C.; Wan, S.; Evtimov, I.; Gabi, D.; Song, D.; Ahmad, F.; Aschermann, C.; Fontana, L.; Frolov, S.; Giri, R. P.; Kapil, D.; Kozyrakis, Y.; LeBlanc, D.; Milazzo, J.; Straumann, A.; Synnaeve, G.; Vontimitta, V.; Whitman, S.; and Saxe, J. 2023. Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models. *arXiv preprint arXiv:2312.04724*.

Cao, Z.; Deng, J.; Yu, L.; Zhou, W.; Liu, Z.; Ding, B.; and Zhao, H. 2026. Remember Me, Refine Me: A Dynamic Procedural Memory Framework for Experience-Driven Agent Evolution. In *Findings of the Association for Computational Linguistics: ACL 2026*, 16803–16822.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H. P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating Large Language Models Trained on Code. In *arXiv preprint arXiv:2107.03374*.

Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations*.

Chen, X.; Tao, Z.; Zhang, K.; Zhou, C.; Zhang, X.; Gu, W.; He, Y.; Zhang, M.; Cai, X.; Zhao, H.; and Jin, Z. 2025. Revisit Self-Debugging with Self-Generated Tests for Code Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 18003–18023.

Comanici, G.; Bieber, E.; Schaekermann, M.; Pasupat, I.; Sachdeva, N.; Dhillon, I.; et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261.

DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348.

Hasan, M. S.; Chakraborty, S.; Karmaker, S.; and Balasubramanian, N. 2025. Teaching an Old LLM Secure Coding: Localized Preference Optimization on Distilled Preferences. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 26039–26057.

He, J.; and Vechev, M. 2023. Large Language Models for Code: Security Hardening and Adversarial Testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 1865–1879.

He, J.; Vero, M.; Krasnopolska, G.; and Vechev, M. 2024. Instruction Tuning for Secure Code Generation. arXiv:2402.09497.

Huang, D.; Zhang, J. M.; Luck, M.; Bu, Q.; Qing, Y.; and Cui, H. 2023. AgentCoder: Multi-Agent-Based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010.

Klemmer, J. H.; Horstmann, S. A.; Patnaik, N.; Ludden, C.; Jr., C. B.; Powers, C.; Massacci, F.; Rahman, A.; Votipka, D.; Lipford, H. R.; Rashid, A.; Naiakshina, A.; and Fahl, S. 2024. Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2726–2740. Association for Computing Machinery.

Li, J.; Le, H.; Zhou, Y.; Xiong, C.; Savarese, S.; and Sahoo, D. 2025. CodeTree: Agent-Guided Tree Search for Code Generation with Large Language Models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 3711–3726. Association for Computational Linguistics.

Liu, A.; Li, H.; Wang, B.; Yang, A.; and Li, H. 2025. RA-Gen: A Controllable Code Generation Framework Using ReAct for Multi-Agent Task Execution. arXiv:2510.08665.

Nie, Y.; Wang, Z.; Yang, Y.; Jiang, R.; Tang, Y.; Davies, X.; Gal, Y.; Li, B.; Guo, W.; and Song, D. 2025. SECODEPLT: A Unified Benchmark for Evaluating the Security Risks and Capabilities of Code GenAI. In *Advances in Neural Information Processing Systems*, volume 38.

Nunez, A.; Islam, N. T.; Jha, S. K.; and Najafirad, P. 2024. AutoSafeCoder: A Multi-Agent Framework for Securing LLM Code Generation through Static Analysis and Fuzz Testing. arXiv:2409.10737.

OpenAI. 2025. Introducing GPT-5.1 for Developers. OpenAI.

Pearce, H.; Ahmad, B.; Tan, B.; Dolan-Gavitt, B.; and Karri, R. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy*, 754–768.

Peng, J.; Cui, L.; Huang, K.; Yang, J.; and Ray, B. 2025. CWEval: Outcome-Driven Evaluation on Functionality and Security of LLM Code Generation. arXiv:2501.08200.

Shin, J.; and Nam, J. 2021. A Survey of Automatic Code Generation from Natural Language. *Journal of Information Processing Systems*, 17(3): 537–555.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 36, 8634–8652.

Siddiq, M. L.; and Santos, J. C. S. 2022. SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques. In *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 29–33.

SWE-agent Team. 2025. mini-SWE-agent: The Minimal AI Software Engineering Agent. Software artifact. Accessed July 25, 2026.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*.

Wang, J.; Cao, L.; Luo, X.; Cao, Y.; Li, Z.; Xiao, Y.; Zhao, M.; Jatowt, A.; and Cai, Y. 2026. Toward Secure Code Generation: Bridging Correctness and Security via Task-Adaptive Vulnerability Modeling and Execution-Based Benchmarking. *Proceedings of the ACM on Software Engineering*, 3(ISSTA): 1–22.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, volume 35, 24824–24837.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K. R.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, volume 37.

Zhang, B.; Du, T.; Tong, J.; Zhang, X.; Chow, K.; Cheng, S.; Wang, X.; and Yin, J. 2024. SecCoder: Towards Generalizable and Robust Secure Code Generation. arXiv:2410.01488.

Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.-J.; and Huang, G. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 19632–19642.